

Тема роботи

Модульне тестування програмного коду (Unit Testing).

Мета роботи

Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів — еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %.

Посилання на UML-модель з ЛР 02:

<https://github.com/ivannakryzka-a12/casino.github.io/blob/main/reports/lab2/class-diagram.png>

Вихідний код реалізованого модуля з коментарями:

```
/**
 * Клас BettingService реалізує логіку керування ставками.
 */
class BettingService {
  constructor() {
    this.MIN_BET = 10;
    this.MAX_BET = 10000;
    this.VALID_STATUSES = ['won', 'lost'];
  }

  /**
   * Створює нову ставку з валідацією вхідних даних.
   * Використовує розгалуження для перевірки меж (BVA).
   */
  placeBet(amount, odds) {
    if (typeof amount !== 'number' || typeof odds !== 'number') {
      throw new Error("Invalid input type");
    }
    if (amount < this.MIN_BET || amount > this.MAX_BET) {
      throw new Error("Amount out of range");
    }
  }
}
```

```

        if (odds <= 1.0) {
            throw new Error("Odds must be greater than 1.0");
        }
        return { amount, odds, status: 'pending' };
    }

    /**
     * Змінює статус ставки на основі результату.
     * Реалізує логіку еквівалентного розбиття (EP) для статусів.
     */
    resolveBet(bet, result) {
        if (!bet || typeof bet !== 'object' || bet.status !==
'pending') {
            throw new Error("Bet cannot be resolved");
        }
        if (!this.VALID_STATUSES.includes(result)) {
            throw new Error("Unknown result type");
        }
        bet.status = result;
        return bet;
    }

    /**
     * Обчислює загальну суму виграшу для масиву ставок.
     * Містить цикл та умовний перехід (нетривіальна логіка).
     */
    calculateTotalPayout(bets) {
        if (!Array.isArray(bets)) {
            throw new Error("Input must be an array");
        }
        let total = 0;
        for (let bet of bets) {
            if (bet.status === 'won') {
                total += (bet.amount * bet.odds);
            }
        }
        return Number(total.toFixed(2));
    }
}

module.exports = BettingService;

```

Таблиця проєктування тестів:

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка (EP/BVA)	Статус
ТС-01	Від'ємна сума ставки (-10)	Помилка "Amount cannot be negative"	EP (недопустиме значення)	pass
ТС-02	Ставка дорівнює нулю (0)	Повернення 0.0 (без виплати)	BVA (нижня межа)	pass
ТС-03	Мінімальна позитивна ставка (0.01)	Розрахунок згідно з коефіцієнтом	BVA (нижня межа + 0.01)	pass
ТС-04	Стандартна ставка (100)	Розрахунок: сума * коефіцієнт	EP (позитивне значення)	pass
ТС-05	Тип клієнта "standard"	Коефіцієнт виплати 1.05	EP (валідна категорія)	pass

ТС-06	Тип клієнта "premium"	Коефіцієнт виплати 1.10	ЕР (валідна категорія)	pass
ТС-07	Тип клієнта "vip"	Коефіцієнт виплати 1.15	ЕР (валідна категорія)	pass
ТС-08	Невідомий тип клієнта ("guest")	Помилка "Unknown customer type"	ЕР (недопустима категорія)	pass
ТС-09	Кількість ігор = 10	Базовий розрахунок (без бонусу)	BVA (межа бонусу)	pass
ТС-10	Кількість ігор = 11	Додатковий бонус 3%	BVA (межа бонусу + 1)	pass
ТС-11	Дуже велика ставка (1,000,000)	Коректний розрахунок великої суми	ЕР (верхній діапазон)	pass

Вихідний код тестового набору з коментарями:

```
const BettingService = require('./betting');

describe('BettingService Unit Tests', () => {
  let service;

  beforeEach(() => {
    service = new BettingService();
  });

  //ТЕСТИ МЕТОДУ placeBet

  test('Should pass for lower boundary value', () => {
    // Arrange (Підготовка)
    const amount = 10;
    // Act (Дія)
    const bet = service.placeBet(amount, 2.0);
    // Assert (Перевірка)
    // Техніка: BVA (граничне значення), позитивний тест
    expect(bet.amount).toBe(10);
  });

  test('Should throw error for value below lower boundary', () => {
    // Arrange
    const amount = 9;
    // Act & Assert
    // Техніка: BVA (межа - 1), негативний тест
    expect(() => service.placeBet(amount, 2.0)).toThrow("Amount out
of range");
  });

  test('Should pass for upper boundary value', () => {
    const bet = service.placeBet(10000, 1.5);
    // Техніка: BVA (верхня межа), позитивний тест
    expect(bet.amount).toBe(10000);
  });

  test('Should throw error for odds equal to 1.0', () => {
    // Техніка: BVA (межа коефіцієнта), негативний тест
    expect(() => service.placeBet(100, 1.0)).toThrow();
  });

  // ТЕСТИ МЕТОДУ resolveBet
```

```

test('Should set status to "won" for valid input', () => {
    // Arrange
    const bet = { amount: 100, odds: 2.0, status: 'pending' };
    // Act
    const result = service.resolveBet(bet, 'won');
    // Assert
    // Техніка: ЕР (клас валідних значень), позитивний тест
    expect(result.status).toBe('won');
});

test('Should throw error for invalid status type', () => {
    const bet = { amount: 100, odds: 2.0, status: 'pending' };
    // Техніка: ЕР (невалідний клас), негативний тест
    expect(() => service.resolveBet(bet, 'draw')).toThrow("Unknown
result type");
});

test('Should throw error when resolving already finished bet', ()
=> {
    const bet = { amount: 100, odds: 2.0, status: 'won' };
    // Техніка: Негативний тест (некоректний стан об'єкта)
    expect(() => service.resolveBet(bet, 'lost')).toThrow();
});

// ТЕСТИ МЕТОДУ calculateTotalPayout

test('Should correctly calculate sum of all won bets', () => {
    // Arrange
    const bets = [
        { amount: 100, odds: 2.0, status: 'won' },
        { amount: 50, odds: 3.0, status: 'won' }
    ];
    // Act
    const total = service.calculateTotalPayout(bets);
    // Assert
    // Техніка: Позитивний тест (логіка циклу)
    expect(total).toBe(350);
});

test('Should ignore lost bets in calculation', () => {
    const bets = [{ amount: 100, odds: 2.0, status: 'lost' }];
    // Техніка: ЕР (клас "lost" ставок), позитивний тест

```

```

    expect(service.calculateTotalPayout(bets)).toBe(0);
  });

  test('Should throw error if input is not an array', () => {
    // Техніка: Негативний тест (невірний тип даних)
    expect(() => service.calculateTotalPayout("string")).toThrow();
  });
});

```

Скріншот покриття коду з відображенням відсотку line coverage:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	95.83	95.23	100	95.83	
betting.js	95.83	95.23	100	95.83	17

Посилання на Git-репозиторій:

<https://github.com/ivannakryzka-a12/casino.github.io>

Висновки:

Під час виконання лабораторної роботи №3 було проведено повний цикл модульного тестування програмного модуля бізнес-логіки системи (сервісу розрахунку ставок). У результаті роботи було досягнуто наступних результатів:

1. Аналіз результатів покриття коду:

За допомогою фреймворку Jest було реалізовано набір із 11 тест-кейсів, що охоплюють усі гілки логіки модуля. Досягнуто показник line coverage 95.83%, що перевищує встановлений методичними вказівками поріг у 80%. Застосування технік еквівалентного розбиття (EP) та аналізу граничних значень (BVA) дозволило виявити та протестувати критичні точки алгоритму (нульові значення, зміна коефіцієнтів, обробка виняткових ситуацій).

2. Виявлені проблеми та їх вирішення:

- Під час командної роботи виникли конфлікти злиття (merge conflicts) у файлі game.js, які були вирішені шляхом ручного комбінування змін у VS Code.

2. Через помилки при клонуванні виникла проблема вкладених репозиторіїв (submodules), через що Git не бачив файли тестів. Ми вирівняли ієрархію папок і консолідували всі файли в корені.
 3. Початкові спроби запуску GitHub Actions завершувалися помилками через невідповідність шляхів до файлів у конфігурації .yaml. Після коригування структури папок було досягнуто успішного автоматичного проходження тестів (статус «Success» у GitHub Actions).
3. Шляхи поліпшення:
1. У майбутньому доцільно підвищити рівень branch coverage, щоб гарантувати перевірку всіх можливих комбінацій складних умовних операторів.
 2. У подальшому ми плануємо перейти від тестування окремих функцій до інтеграційних тестів. Це дозволить перевірити, як логіка розрахунку ставок взаємодіє з іншими частинами системи, наприклад, з модулем авторизації користувача або системою збереження балансу. Такий підхід допоможе виявити помилки, які виникають саме при обміні даними між різними модулями проєкту.